



DASHVANTI

Project and Operations Runbook

Architecture • Modules • Navigation • Database • Deployment

A practical guide to the customer, restaurant, driver and administrator portals, with live interface screenshots and workflow diagrams.

Prepared from the deployed source, installed runtime packages, registered API routes and the live Oracle schema export. Screenshots are taken from the running website with personal details masked.

Document version 1.0 | 13 September 2026 UTC

Repository: <https://github.com/KB6061/Dashvanti>

Contents

01 Operator quick reference	3
02 System architecture	4
03 Technologies and runtime	5
04 Application modules	6
05 Customer workflow	7
06 Restaurant workflow	11
07 Driver navigation and assignment	15
08 Tracking and notification flow	18
09 Cancellation and refund rules	19
10 Administrator operations	20
11 Deployment and service operations	23
12 Configuration and secrets	24
13 Monitoring and troubleshooting	25
14 Backup, restore and disaster recovery	26
15 Release, rollback and validation	27
16 Security and production readiness	28
17 Data relationships and model catalog	29
18 API inventory and source references	39

Screenshot plates follow the relevant workflow sections. The model catalog includes all 25 tables; the API appendix lists the registered routes.

01 Operator quick reference

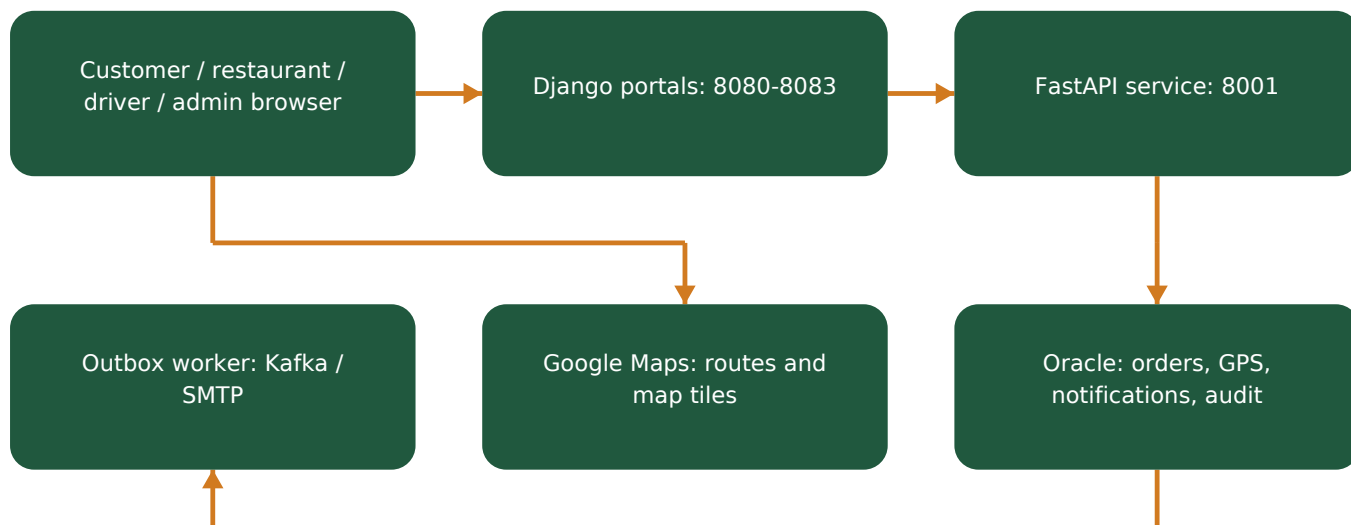
Dashvanti is a four-portal food delivery application. This runbook documents the running deployment and source snapshot inspected on 13 September 2026 UTC. It is an operating reference, not a claim of production certification.

Application root: /home/krishna/food. Repository: github.com/KB6061/Dashvanti. Database owner: DASHVANTI. The private API is reached by Django; browser clients use their own portal endpoints.

Component	Observed address / command	Purpose
Customer	http://192.168.56.101:8080/customer/login	Browse, checkout, order tracking, receipts
Restaurant	http://192.168.56.101:8081/restaurant/login	Catalog, preparation, order history
Driver	http://192.168.56.101:8082/driver/login	Availability, GPS, delivery, earnings
Admin	http://192.168.56.101:8083/admin/login	Operations, dispatch, funds and oversight
Internal API	http://127.0.0.1:8001/health	Health endpoint; business routes use /api
Service control	bash script/dashvanti-services.sh status	Inspect the five application processes

- Start an incident by identifying the order ID, order mode, current restaurant state, driver state, and timestamp.
- Pickup orders do not enter driver dispatch. Delivery orders require a fresh, eligible driver location and driver acceptance.
- Never paste .env contents, bearer tokens, password-reset links, customer addresses, or raw location histories into support tickets.

02 System architecture



Django renders HTML and proxies authenticated requests to FastAPI. API bearer tokens are retained in server-side file sessions; cookies identify the session. SQLAlchemy services enforce ownership, transitions and transactional writes in Oracle.

The maps use Google Maps JavaScript and Directions services. Backend route estimates also use Google routing services. Notifications are polled from the application; Kafka is not required for the browser polling path.

A Kafka broker process was visible during inspection. No backend.worker process was visible in that process snapshot. Outbox publishing and password-reset email delivery must be checked separately; broker availability alone does not prove events are delivered.

- Business logic: backend/services; HTTP routes: backend/routers; input validation: backend/schemas.py.
- Persistence models: backend/models.py. Actual Oracle DDL: database/oracle_schema.sql.
- Portal templates: frontend/templates and each role's templates folder. Shared browser behavior: frontend/static.
- Deployment scripts and alternate container scaffolding coexist. The inspected running deployment uses direct Python processes, not Compose.

03 Technologies and runtime

Versions below were read from installed package metadata on the server. requirements.txt declares version ranges; requirements.lock.txt is an additional repository artifact and must be checked before using it to recreate this runtime.

The observed python-oracledb version is 4.0.2, while requirements.txt specifies oracledb >=2.5,<4. This is a reproducibility gap: reconcile the dependency files and validate the chosen environment before a fresh installation.

Layer	Technology	Responsibility
UI	HTML, CSS, browser JavaScript	Responsive portal screens, dialogs, tables, map markers
Web tier	Django	Templates, sessions, CSRF protection, API proxy
API	FastAPI, Pydantic, Uvicorn	Validated role-scoped endpoints
Data	Oracle, SQLAlchemy, python-oracledb	Transactions, row locking, relational persistence
Authentication	PyJWT, Argon2 password hashing	API sessions and password verification
Mapping	Google Maps / Directions / Routes	Pins, polylines, distance, ETA
Images / PDF	Pillow, ReportLab	Image processing, receipt/report PDFs
Integration	Kafka, SMTP, HTTPX	Outbox publishing, email, service calls
Verification	pytest	Workflow, permissions, dispatch and tracking tests

Component	Observed version
Python	3.10.14
fastapi	0.141.1
uvicorn	0.52.4
SQLAlchemy	2.0.52
oracledb	4.0.2
Django	5.2.17
PyJWT	2.13.0
pydantic	2.13.4
httpx	0.28.1
Pillow	12.3.0
reportlab	5.0.1
confluent-kafka	2.15.1

04 Application modules

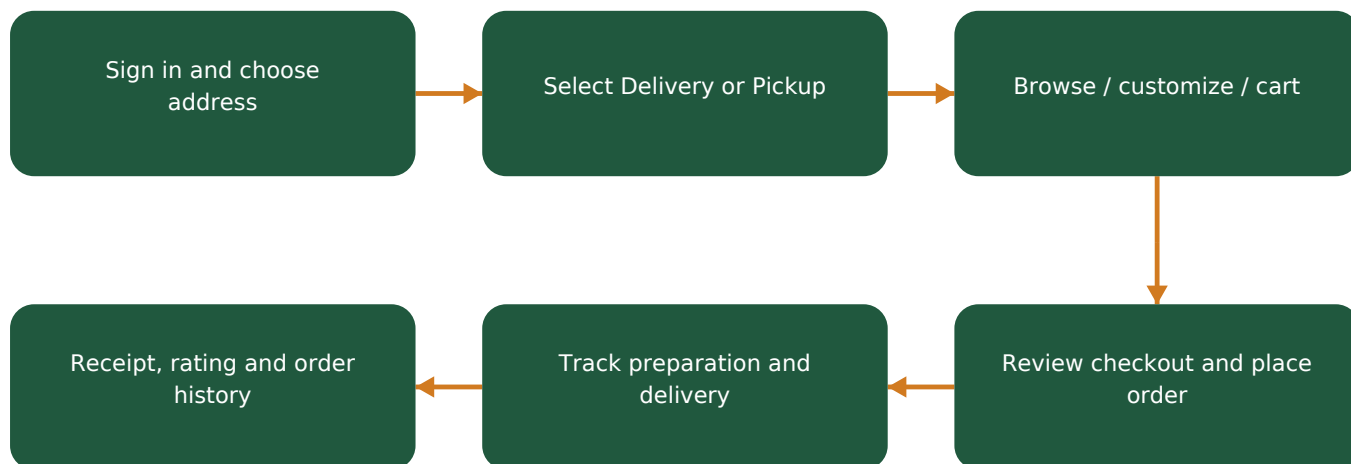
Authentication flow: browser form -> Django CSRF validation -> FastAPI authentication service -> Argon2 password verification -> JWT stored in the server-side session. Browser cookies identify that session rather than exposing the bearer token.

Receipt/report flow: authorized order or earnings request -> report service -> ReportLab PDF -> in-site preview -> Download or Print. Receipt dates and times are displayed using the application's local-time presentation requirements; verify the configured timezone when moving servers.

Upload flow: authenticated request -> file-service validation and image processing -> private filesystem object -> Oracle FILES metadata. Thumbnail endpoints reduce menu-list image transfer; original files remain access-controlled.

Module	Main source	Main behavior
Authentication and profiles	auth_service.py; user_service.py	Role login, account data, reset flow, addresses
Restaurant catalog	restaurant_service.py; menu_service.py	Restaurant details, menus, menu availability
Store availability	store_status_service.py	Open/closed and paused ordering checks
Cart and checkout	order_service.py	Cart validation, pricing, order snapshot, idempotency
Preparation and delivery	delivery_service.py	Restaurant and driver transitions, order ownership
Dispatch	dispatch_service.py	Fresh GPS, 3-mile eligibility, first acceptance, reassignment
Live tracking	tracking_service.py; navigation_service.py	Driver location, separate states, route and ETA response
Arrival detection	arrival_service.py	Customer arrival event near destination
Cancellation	cancellation_service.py	Preview, fees, refunds, release and policy snapshot
Funds and promotions	fund_service.py; operations_service.py	Admin rules, refund requests, operational controls
Reports and receipts	report_service.py	Receipt previews and downloadable reports
Files	file_service.py	Private uploads, normalized images, thumbnail responses
Notifications / audit	operations_service.py	User notifications and operational audit trail
Integration outbox	kafka_event_service.py; backend/worker.py	Transactional events and retrying external delivery

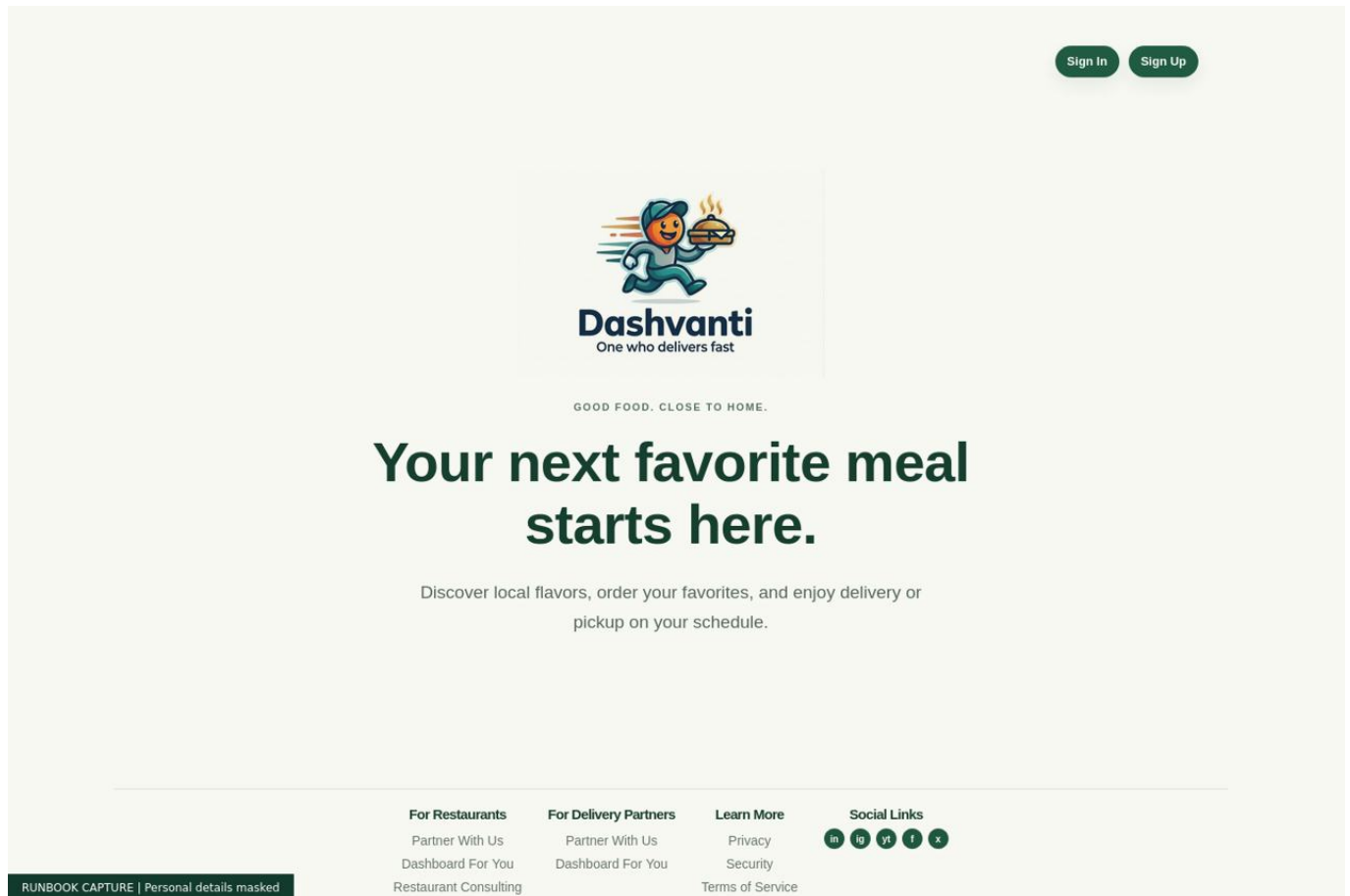
05 Customer workflow



- Choose an address from map suggestions; save a label and optionally set it as default. Confirm the address and delivery/pickup mode at checkout.
- A closed or paused store must not accept a new order. The closed-store dialog offers browsing and alternative open stores.
- Order placement stores item prices and totals. An existing order is not repriced simply because the administrator changes a fee rule.
- For delivery, observe restaurant activity separately from driver travel. For pickup, the restaurant completes the pickup workflow and no driver is assigned.
- The customer's order sound is limited to the arrival event near the customer's location. It is not a continuous preparation or status alarm.
- View order bill opens the receipt preview with Print and Download. A delivered order exposes Rate and review.

Customer entry page

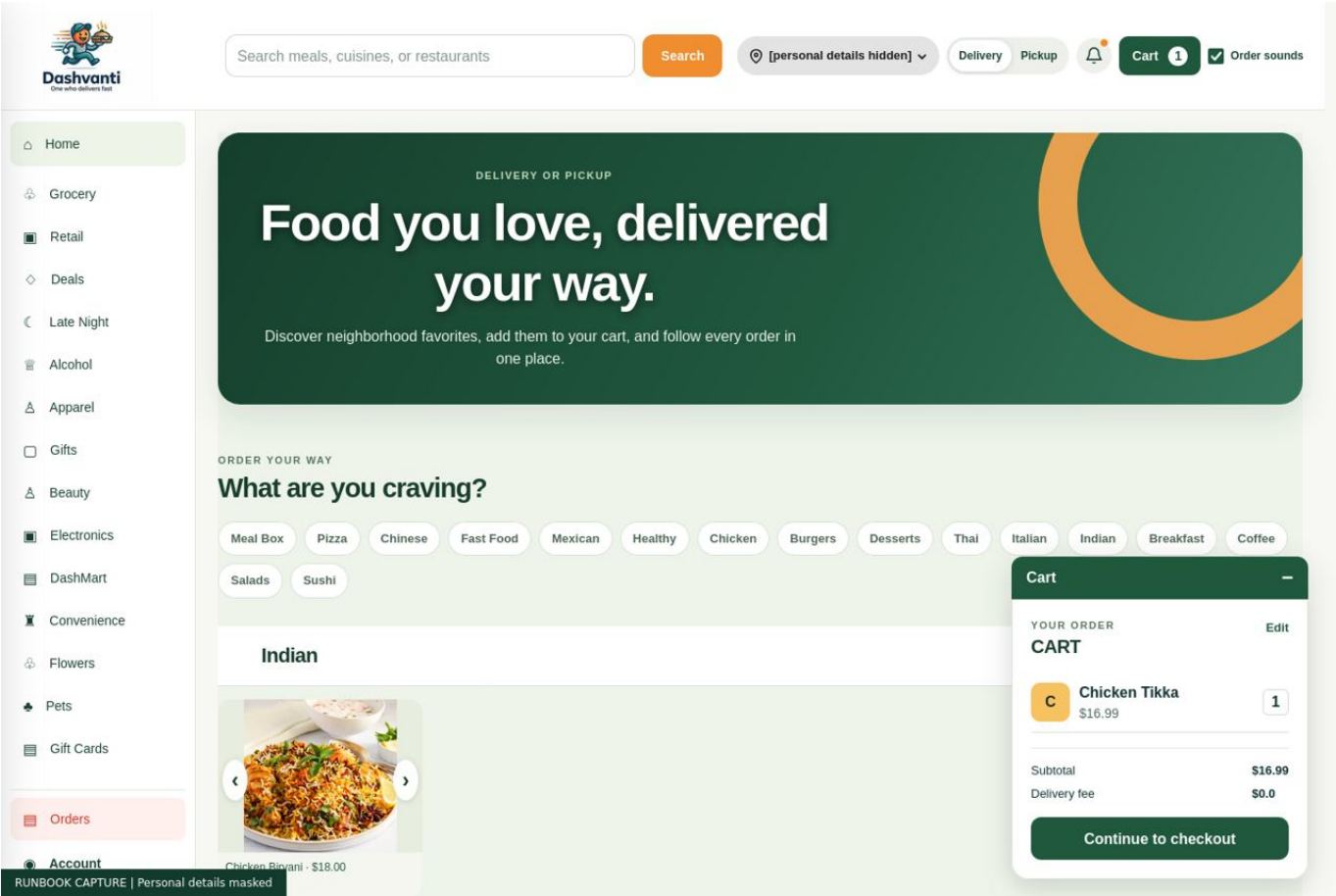
Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Customer browsing dashboard

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Customer order history

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Search meals, cuisines, or restaurants

Search

[personal details hidden]

▼

Delivery

Pickup

🔔

Order sounds

Cart

0

Home

Grocery

Retail

Deals

Late Night

Alcohol

Apparel

Gifts

Beauty

Electronics

DashMart

Convenience

Flowers

Pets

Gift Cards

Orders

Account

Back to dashboard

Orders

All orders

Active

Completed

Cancelled

Sep 12, 2026, 6:44 PM

[personal details hidden]

Order #27 · Delivered

\$51.98 · 3 items · Pickup

Aloo Tamatar Soup · Chicken Biryani · Chicken Tikka

Reorder

View order bill

Sep 12, 2026, 5:25 PM

[personal details hidden]

Order #26 · Cancelled

\$83.47 · 4 items · Delivery

Chicken Biryani · Palak Paneer · Chicken Tikka · Methi Malai Chicken

Reorder

View order bill

Sep 12, 2026, 4:06 PM

[personal details hidden]

Order #25 · Delivered

\$74.46 · 6 items · Delivery

Aloo Tamatar Soup · Chicken Biryani

Reorder

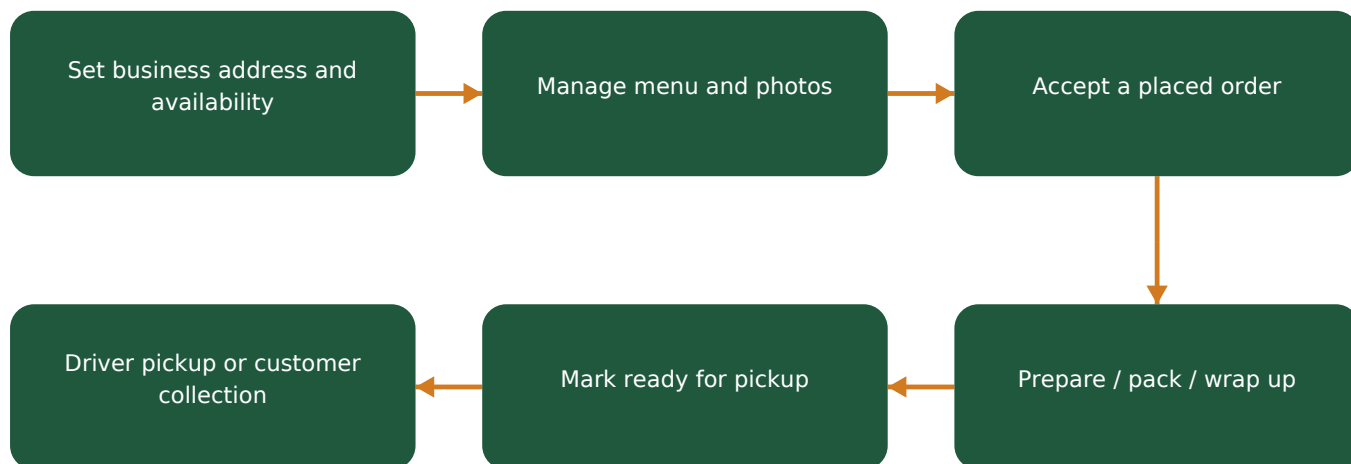
View order bill

Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Internal operations reference | Definitions and masked screenshots only

10

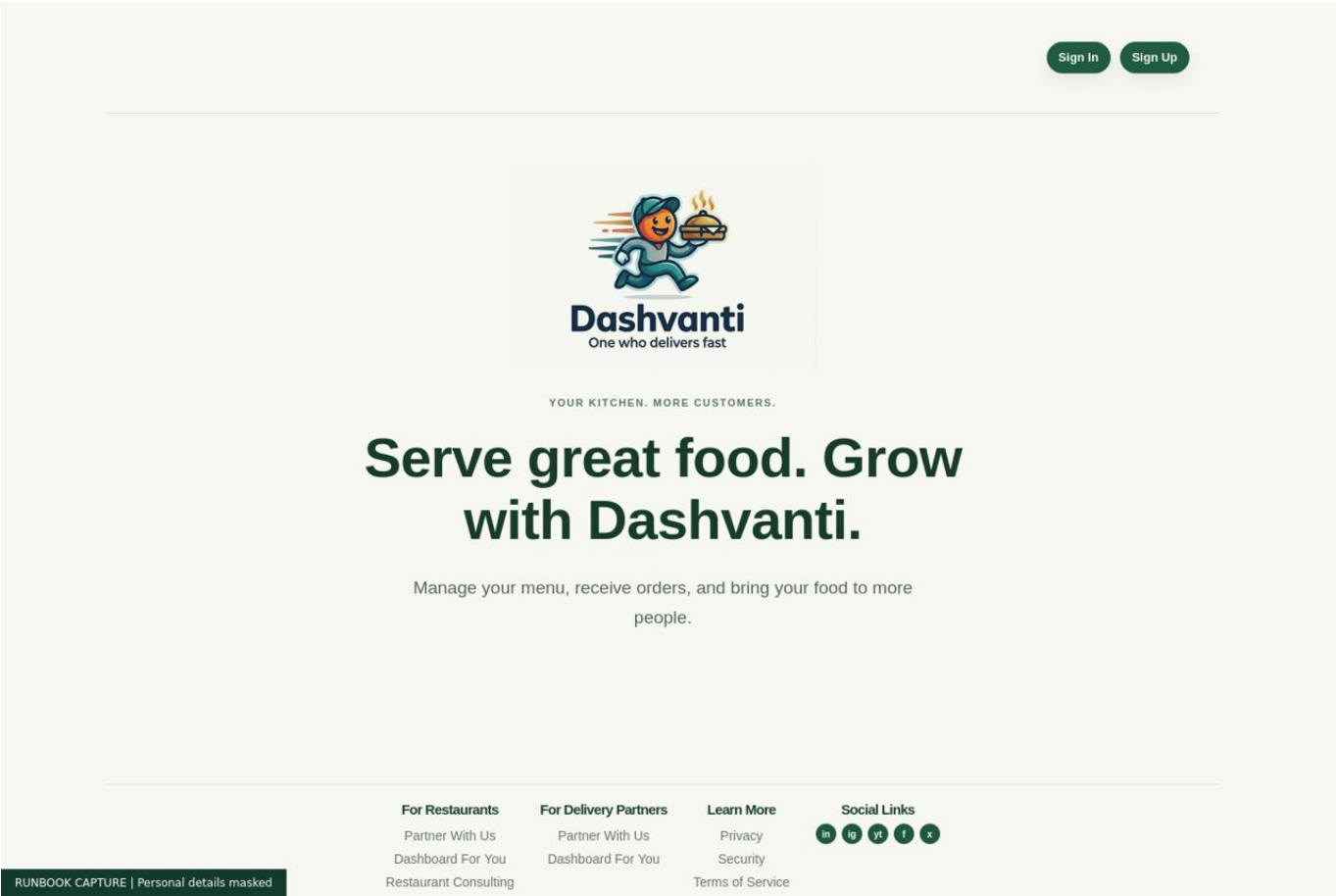
06 Restaurant workflow



- Check the saved business address and map pin before opening the store. Opening hours are presented to users; the availability controls remain important to order acceptance.
- Accepting a delivery order makes it eligible for nearby-driver offers. A pickup order does not trigger delivery dispatch.
- Preparation states include ACCEPTED, PREPARING, PACKING, WRAPPING_UP and READY_FOR_PICKUP. Driver travel is recorded separately.
- Do not mark food ready before it is ready. Driver pickup is guarded by readiness and delivery transition rules.
- Search historical orders by supported identifiers or customer/menu terms. Open order details for line items and timeline; use the receipt preview for the bill.
- Restaurant cancellation before pickup requests a full remaining refund. After pickup, escalate for admin review.

Restaurant entry page

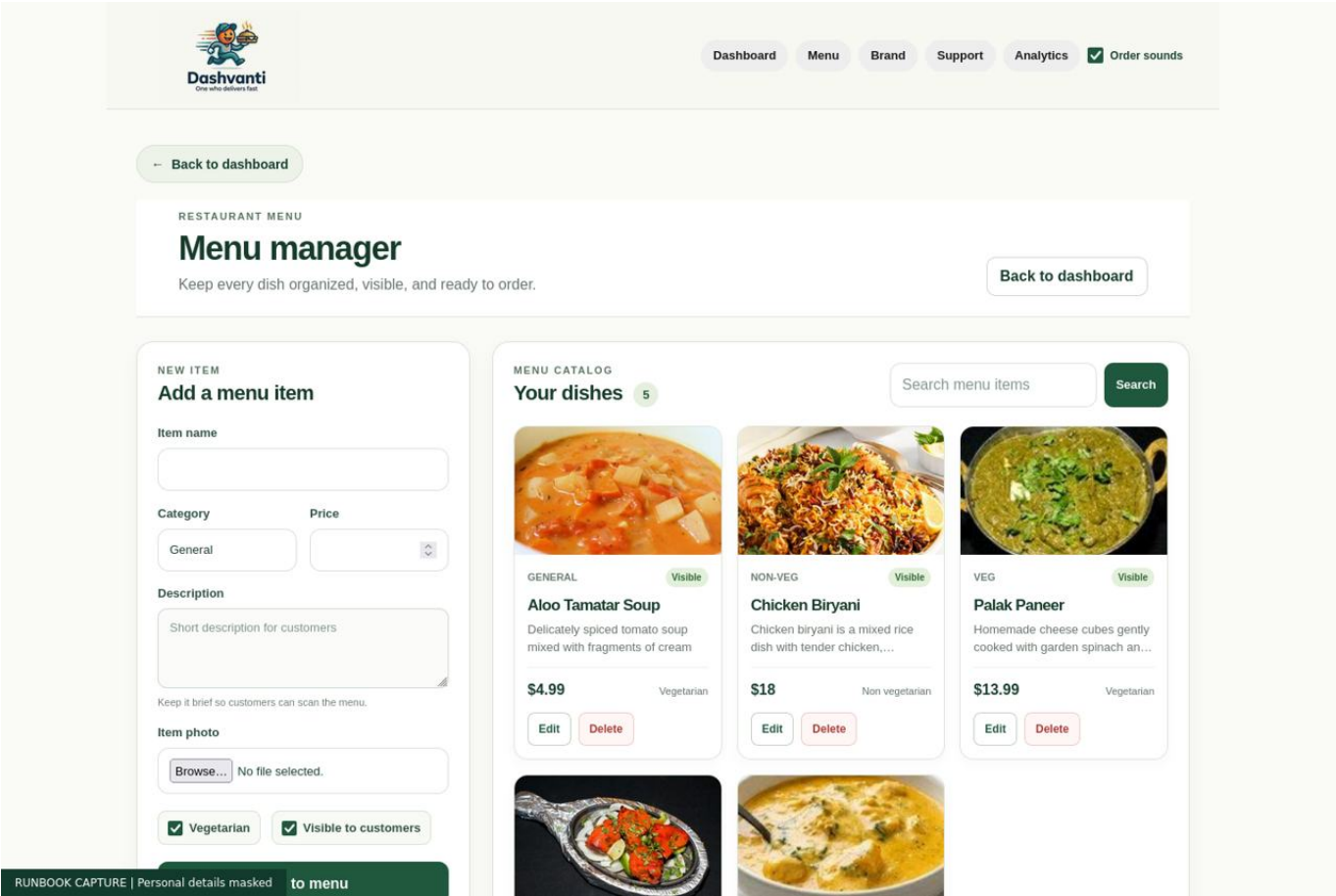
Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Restaurant menu manager

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Restaurant order history

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



[Dashboard](#)[Menu](#)[Brand](#)[Support](#)[Analytics](#)[Order sounds](#)

[Back to dashboard](#)

Orders

5 results

Search all orders

Customer name, phone, order number or menu item

Status

All statuses

Order ID	Date	Customer	Driver	Mode	Total	Status
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]	[redacted]

[Previous](#)5 orders · Page 1 of 1[Next](#)

For Restaurants

Partner With Us

Dashboard For You

Restaurant Consulting

For Delivery Partners

Partner With Us

Dashboard For You

Learn More

Privacy

Security

Terms of Service

Social Links

[in](#)[ig](#)[yt](#)[f](#)[x](#)

RUNBOOK CAPTURE | Personal details masked

Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

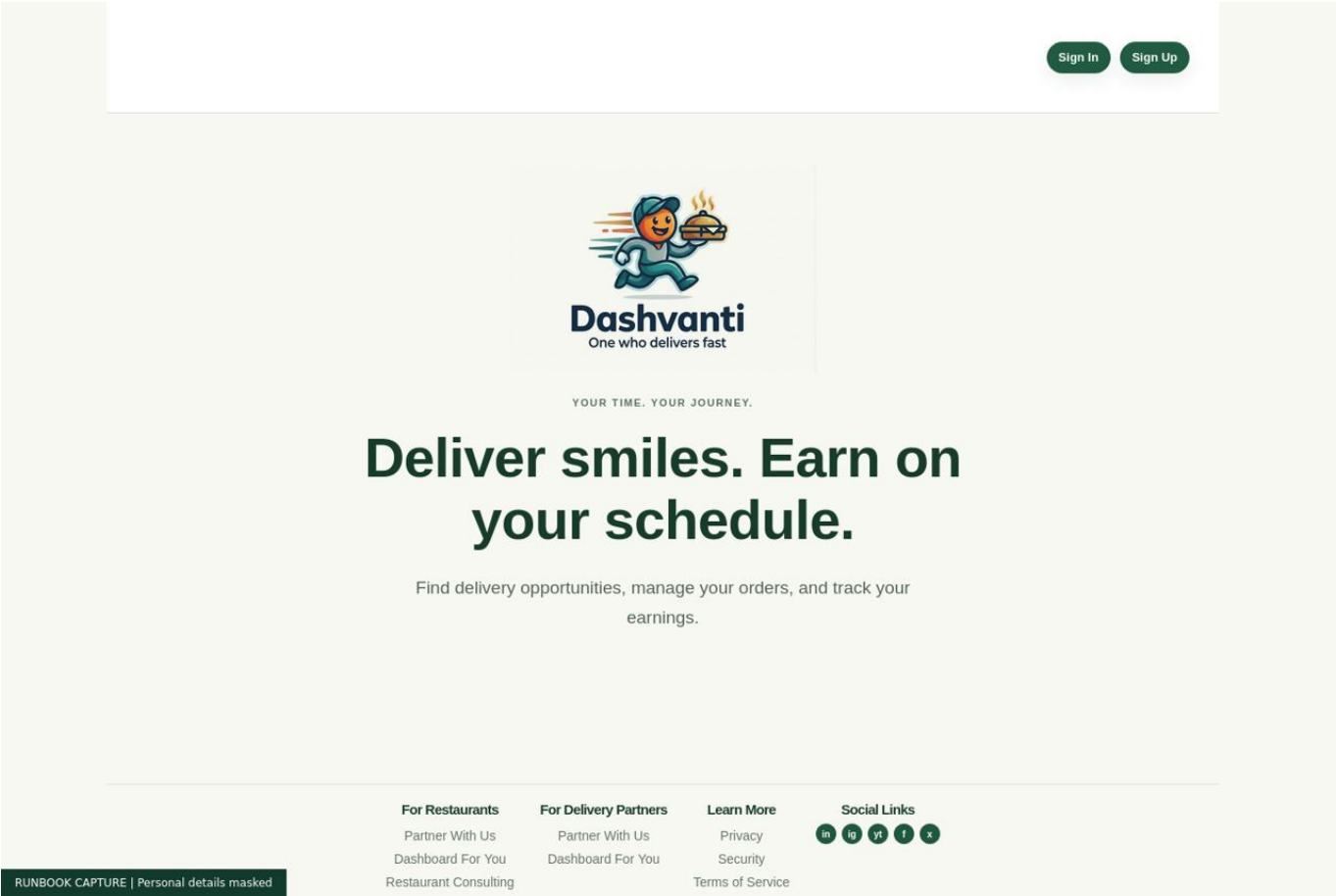
07 Driver navigation and assignment



- Location permission requires a secure browser context. The deployed LAN HTTP URL requires an HTTPS rollout for normal operation. A local Chrome testing exception does not encrypt traffic.
- GPS is sampled and sent on a three-second schedule while the portal is active. Browser suspension, permission denial, lack of signal or network failure can delay updates; this is not a native background tracking service.
- Eligibility requires online status, GPS updated within two minutes, no other active delivery, a valid restaurant address and a driving route within three miles.
- The first eligible driver to accept claims the order under a row lock. Other offered drivers receive the claimed notification. Admin can reassign an eligible nearby driver before pickup.
- Use Start journey, Arrived at restaurant, Picked up order, Start delivery and Delivered in sequence. Pickup is blocked until food is READY_FOR_PICKUP.
- The red car marker remains visible when stationary. Directions, pins, distance and ETA are displayed with the route. Check the last GPS timestamp if movement stops.
- Before pickup, release the delivery with a reason if unable to continue. The order remains active for another driver; the releasing driver is excluded from that order's next offers.

Driver entry page

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Driver dashboard

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Dashboard

Track Order

Active Deliveries

Earning Reports

Profile

Support

Dashboard

Track Order

Active Deliveries

Earning Reports

Profile

Support

J

0 Active

0 New Orders

Online

Live GPS

Standby

WELCOME BACK, [personal details hidden]

Power Status

Update

ACTIVE ORDERS & MAPPING

Live location / delivery details hidden for documentation

Loading Google Maps...

Check Equipment

Payment Methods

Vehicle Info

Live location / delivery details hidden for documentation

EARNINGS & PERFORMANCE

TODAY'S EARNINGS

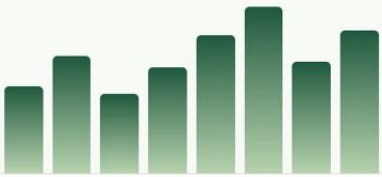
\$13.00

ORDERS COMPLETED

2

RATING

4.8 / 5



ANNOUNCEMENTS & OFFERS

New Area Incentives

Earn extra on evening delivery batches.

Gear Discounts

Save on insulated bags and phone mounts.

Safety Reminders

Keep your route and customer safe.

CHAT & SUPPORT

Order Issue

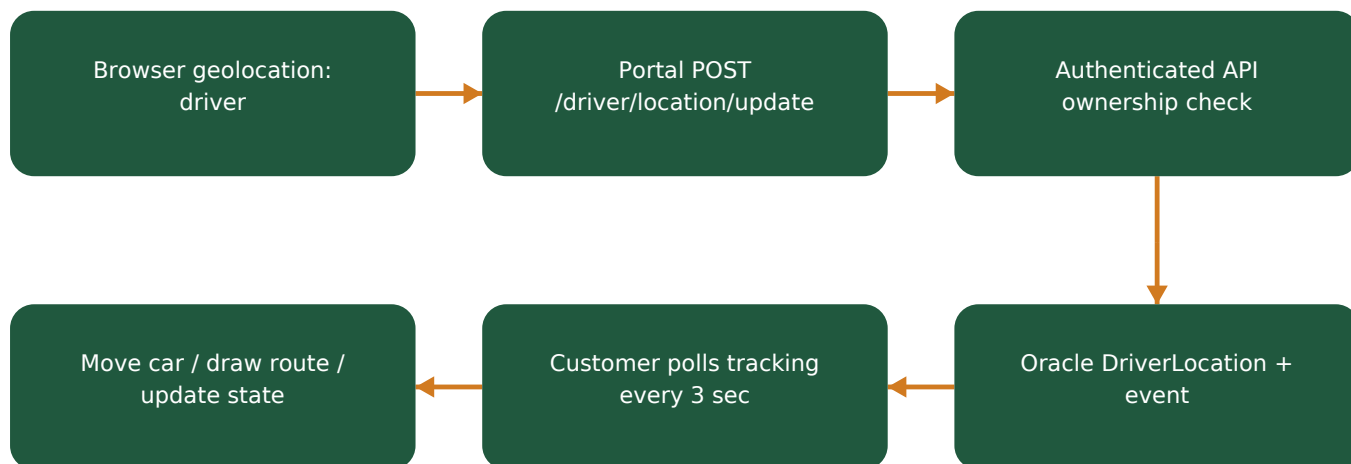
You are assigned to order #26.

Dismiss

RUNBOOK CAPTURE | Personal details masked

Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

08 Tracking and notification flow



Tracking marks a location stale after 30 seconds. Dispatch uses a separate two-minute freshness threshold. Backend navigation results are cached for 30 seconds; temporary failures are cached briefly. ETA is an estimate and must not be displayed as a guaranteed delivery deadline.

Sound preference is persisted per account. Browser audio still needs user interaction to unlock playback. Alerts are one three-second tone, not a repeating alarm. A new assignment has a dedicated driver sound path. The customer arrival event is emitted near the drop-off (50 metres driving-distance threshold in the current implementation).

Completed/cancelled orders must not reveal the driver's later location. Polling uses no-store responses. Route or Google service failures should retain the last known status and present a reconnect/unavailable message.

Endpoint (API prefix /api)	Purpose
POST /driver/location/update	Accept lat/lng or latitude/longitude; validate optional driver/order IDs; save latest GPS
POST /driver/status/update	Validate and record the driver's next status
GET /order/{id}/driver/location	Return assigned driver location for an authorized order viewer
GET /order/{id}/tracking	Return restaurant state, driver state, location freshness, history and route/ETA information
GET /order/{id}/restaurant/status	Return current restaurant activity

09 Cancellation and refund rules

The current default preparation percentage is 100; review threshold is three events in a 30-day window. Treat these as configurable policy values, not permanent business constants. Each order stores a policy snapshot so a later edit cannot silently change that order's terms.

The rollout protected already-existing orders using zero preparation-fee snapshots. New orders use the policy at creation. Preview confirmation checks status and amounts; if the order changes, refresh the preview.

Cancellation is idempotent. Previous refund requests count toward the remaining payment. A refund record is PENDING until payment-provider processing is confirmed; the application does not prove a bank/card transfer occurred.

Repeated cancellations/releases create review signals, not automatic penalties. Reasons and audit records must be retained. Do not manually delete refund entries tied to a cancellation calculation.

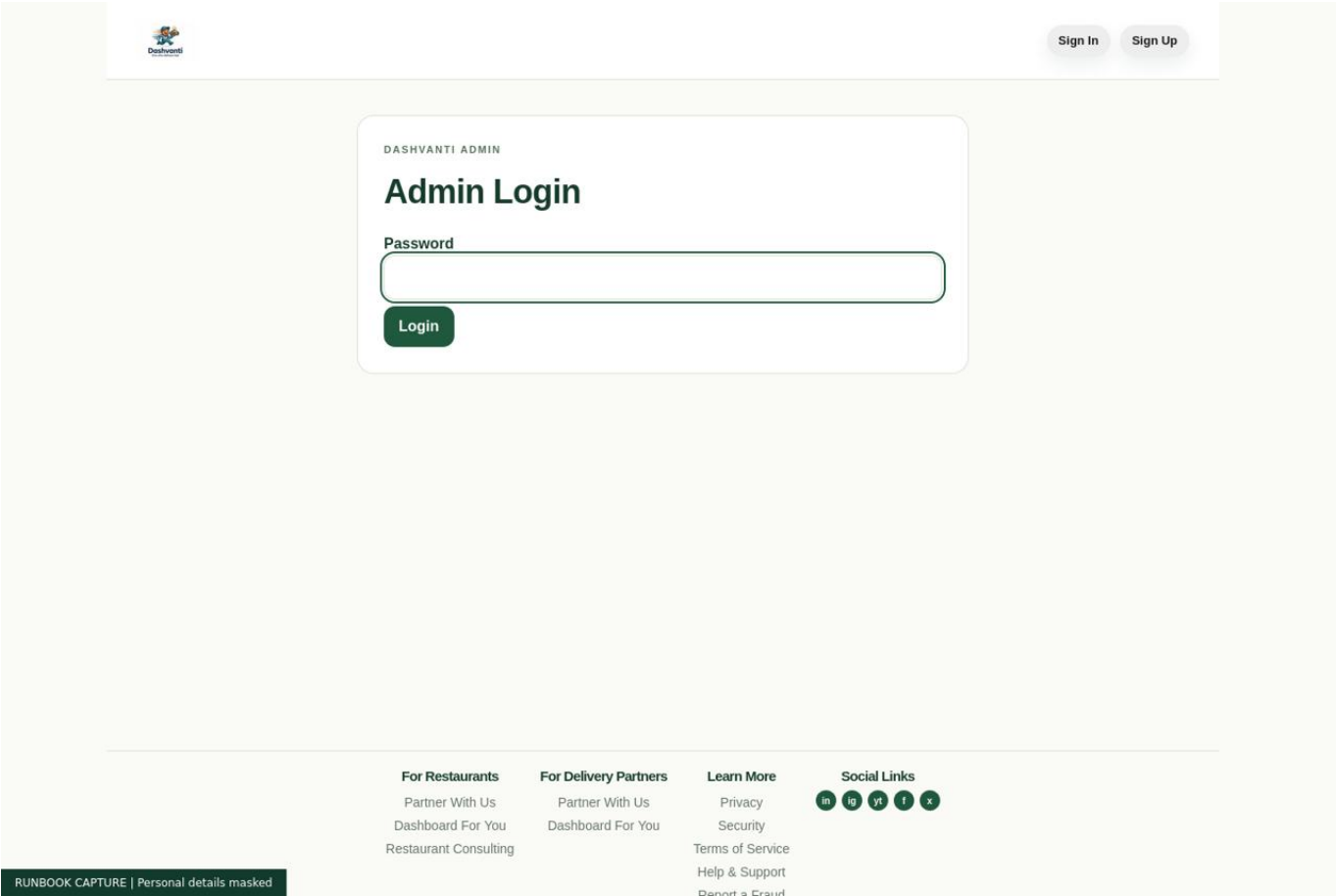
Actor / timing	Order effect	Financial effect
Customer before preparation	Cancel order	Full remaining refund request
Customer during preparation	Cancel after exact preview	Retain configured percentage of discounted food value, capped at remaining payment
Customer after pickup	Self-service cancellation blocked	Admin review required
Restaurant before pickup	Cancel with reason	Full remaining refund request
Driver before pickup	Release assignment; redispach	Does not cancel the customer's order
Driver after pickup	Escalate to support	No ordinary self-release
Admin	Review and cancel; controlled override	Refund cannot exceed remaining paid amount

10 Administrator operations

- Use the admin dashboard to review live orders, restaurants, fleet, customers, promotions, funds and analytics. Permissions and credentials must be restricted to authorized operators.
- Order inspection: confirm mode, preparation state, driver assignment and timeline before changing anything. An unassigned pickup is normal, not a dispatch outage.
- Reassignment: verify the expected current driver, replacement availability, fresh GPS and the three-mile driving-distance rule. Record a reason and confirm affected parties receive updates.
- Funds: adjust pricing for new orders, review cancellation policy and track refund requests. Reconcile the application ledger against the actual payment or cash process.
- For a complaint, use the order ID and authorized contact details in the order screen. Do not copy customer contact information into the runbook or broad operational channels.
- Support and audit records provide traceability. Investigate failures before retrying financial operations.

Administrator sign-in

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.



Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

Administrator fund management

Live portal capture. Personal information is masked; no order or financial actions were performed for the capture.

Back to dashboard

Admin Portal

Dashboard

Orders (Live)

Restaurants

Delivery Fleet

Customers

Promotions

Fund Management

Finances

Analytics

Settings

Search orders, drivers, restaurants...

System Online

Logout

System Admin

Full access

Cancellation policy

Free before preparation. During preparation, the charge is a percentage of discounted food value. Restaurant/admin cancellation gives a full refund of the remaining payment. After pickup, admin review is required. Changes apply to new orders.

Preparation charge (%)

100

Flag for review after cancellations in 30 days

3

Save cancellation policy

Fund Management

Database pricing

Core fee & pricing controls

Changes apply to new checkout totals. Existing orders retain their original charges.

Rule	Calculation	Value	Minimum (USD)	Status	Actions
Delivery fee	Fixed amount	\$5.00	\$2.00	Enabled	Update Delete
Discount	Fixed amount	\$2.00	\$8.00	Enabled	Update Delete
Refund limit	Fixed amount	\$50.00	\$10.00	Enabled	Update Delete
Service fee	Fixed amount	\$10.00	\$15.00	Enabled	Update Delete
	Fixed amount	\$3.44	\$5.00	Enabled	Update Delete

RUNBOOK CAPTURE | Personal details masked

Use the preceding workflow steps to interpret this screen. Screens may change with later releases.

11 Deployment and service operations

These commands describe the repository's direct-service controller. Review the script before use in a new environment. Its API start path calls backend.migrate, and a restart interrupts active portal requests. This is not a zero-downtime deployment mechanism.

The outbox worker entrypoint is `python3 -m backend.worker`. Run it under the approved process supervisor with the same protected application configuration; verify Kafka acknowledgements and SMTP delivery before relying on it.

- Before maintenance: record the release, check active deliveries, back up required data, confirm the rollback plan and notify operations.
- Verify process identities and listening ports before stopping anything. PID files can become stale after manual restarts.
- The observed web processes use `manage.py runserver --insecure --noreload`. Replace development serving with an approved production WSGI/ASGI deployment and trusted HTTPS ingress before public launch.
- Do not assume an API restart starts the outbox worker: the controller manages the API and four portals, not a verified worker service.
- Use a dedicated environment with reconciled dependency versions. Avoid installing application dependencies into a shared server interpreter during a live release.

```
cd /home/krishna/food
bash script/dashvanti-services.sh status
curl -fsS http://127.0.0.1:8001/health
bash script/dashvanti-services.sh logs

# Planned maintenance only:
bash script/dashvanti-services.sh restart driver
bash script/dashvanti-services.sh restart restaurant
bash script/dashvanti-services.sh restart all
```

12 Configuration and secrets

The GitHub upload intentionally excludes live .env files, uploads, log files, sessions and virtual environments. A clone is source code, not a complete operational backup. Supply secrets and persistent storage through the deployment process.

Setting group	Purpose / operating requirement
DATABASE_URL	Oracle connection. Use a secret store or restricted .env; never include its value in documentation.
JWT_SECRET / DJANGO_SECRET_KEY	Strong independent secrets; rotation invalidates or affects sessions and needs a rollout plan.
ADMIN_PASSWORD	Admin access and internal admin authorization. Configure explicitly; never rely on source defaults.
API_URL / portal URLs	Private API base and role-specific public addresses. Match the actual ports/HTTPS origin.
ALLOWED_HOSTS / CSRF_TRUSTED_ORIGINS	Explicit deployed hostnames and secure origins.
COOKIE_SECURE	Enable with HTTPS; verify session and CSRF cookies after the change.
GOOGLE_MAPS_API_KEY	Required map/routing APIs, billing and appropriate key restrictions; use separate browser/server keys when deploying that design.
SESSION_FILE_PATH / FILE_ROOT	Restrict filesystem access, provision persistent storage and back up media.
KAFKA_BOOTSTRAP_SERVERS / SMTP_*	Broker and mail transport; validate worker delivery separately.
PORTAL_ROLE / logging variables	Portal isolation, cookie naming and log destinations.

13 Monitoring and troubleshooting

Symptom	Checks	Operator action
No driver assigned	Pickup vs delivery; restaurant acceptance; online state; GPS age; route distance; active job	Do not bypass eligibility. Correct GPS/address or arrange an eligible replacement.
GPS cannot be enabled	Secure context; browser/OS permission; location service; portal online state	Deploy trusted HTTPS, allow permission and verify a recent backend timestamp.
Driver alert is silent	Order mode; delivery offer/assignment record; saved sound preference; browser AudioContext	Interact with the portal to unlock audio; verify one three-second tone on a controlled test.
Car frozen / ETA missing	GPS timestamp, stale flag, browser backgrounding, network and Google errors	Restore location/network; distinguish last known location from live movement.
Map centered incorrectly	Saved restaurant address, geocoding/routing result, Google API access	Correct the saved address; confirm pin and business name.
Order status update rejected	Ownership, current state, food readiness, conflicting assignment	Refresh and follow the next allowed action; never update status directly in SQL.
Refund pending	Cancellation record, previous refunds and actual provider reconciliation	Process/verify the external transfer through the approved finance workflow.
API unavailable	/health, process, Oracle connectivity, error logs	Restore the failed component; preserve logs and avoid exposing secrets.
Reset email absent	Pending outbox, worker process, SMTP TLS/sender/configuration	Restore worker/mail delivery; avoid logging reset URLs or raw payloads.
Slow menu photos	Thumbnail response/cache, image size, network, server load	Use cached thumbnails and measure requests; no network UI can promise zero latency.

14 Backup, restore and disaster recovery

- Assign an owner, approved backup schedule, retention, recovery-point objective and recovery-time objective. These targets were not verified or agreed by this inspection.
- Back up Oracle using the DBA-approved method, and back up FILE_ROOT media on a coordinated schedule. Store secrets separately with restricted access and encryption.
- database/oracle_schema.sql contains definitions only: 25 tables, 160 columns, primary/unique/check constraints, 15 additional indexes and 31 foreign keys. It does not restore orders, accounts, media, grants or a full database service.
- Test restores into an isolated environment. Provision the Oracle service and a least-privilege schema owner before running DDL. Use an empty schema; do not run the creation script over the live schema.
- For a full restore, follow the DBA's data import procedure, restore matching media, configure secrets, then verify authentication, catalog, sample historical orders and file access.
- Restore external services and workers separately. Reconcile pending refunds and outbox events before resuming processing; downstream events may be delivered more than once.
- Record restore duration, checks, release/schema versions and operator sign-off. A backup is not considered proven until a restore has been exercised.

15 Release, rollback and validation

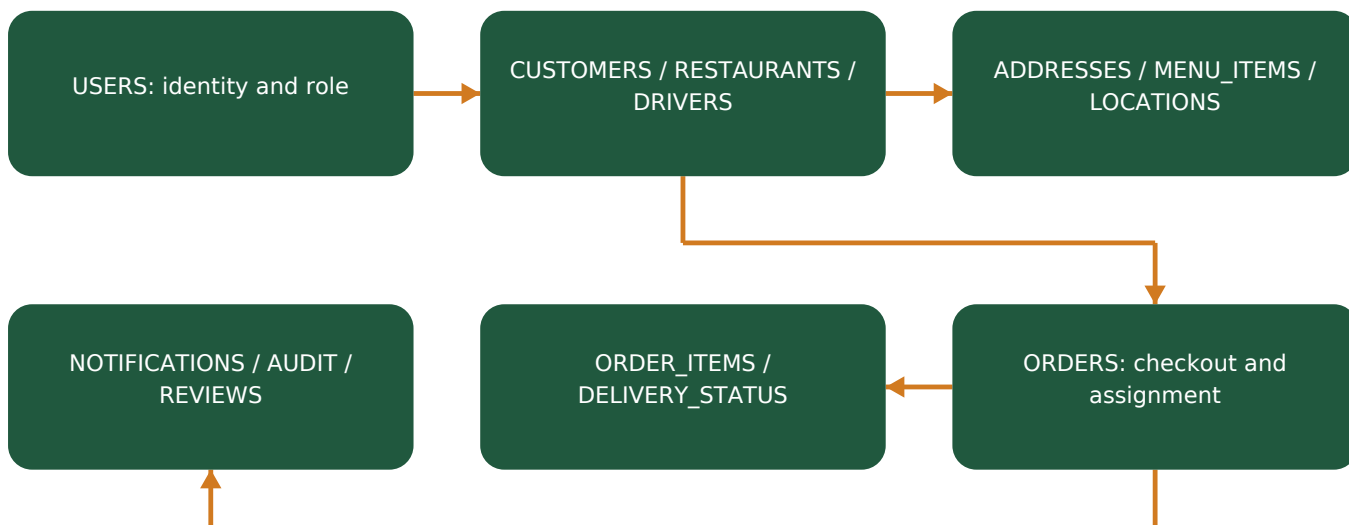
- Run checks in a staging/test environment with the required configuration. API tests use SQLite; they do not establish Oracle lock behavior, provider availability, mobile GPS behavior or production capacity.
- Test one complete delivery: customer checkout; restaurant accept/prepare/ready; eligible driver accept/arrive/pick up/deliver; customer tracking and rating.
- Also test pickup without dispatch, closed-store rejection, stale GPS, simultaneous driver acceptance, sound permission, cancellation previews and duplicate cancellation requests.
- Validate screenshots and responsive layout on desktop and mobile; confirm receipt preview, Download/Print, dates and 12-hour local time.
- Before deployment, record a known-good release and schema backup. Review explicit migration SQL; backend.migrate is bootstrap logic, not a complete versioned upgrade framework.
- For rollback, restore the previous application release and compatible dependencies. Do not reverse or drop database columns blindly; restore or forward-fix through the approved migration plan.
- After release, check all four portals, route/tracking endpoints, logs, notification lag and worker health. Verify live behavior with controlled test accounts rather than changing real customer orders.

```
cd /home/krishna/food
python3 -m pytest -q
python3 frontend/manage.py check
curl -fsS http://127.0.0.1:8001/health
```

16 Security and production readiness

- Enforce trusted HTTPS and private API networking. The current HTTP LAN setup is a staging configuration; browser location access and secure cookies need deliberate HTTPS deployment.
- Browser-delivered HTML, CSS and JavaScript can be inspected. Protect secrets on the server and enforce authorization in the API; disabling right-click/F12 is not source-code protection.
- Role checks, order ownership, CSRF, server-side sessions and row locks exist in the code. This runbook is not a penetration-test report or a concurrency proof.
- Restrict uploads and document access. Keep filesystem data private, review allowed file types and limits, and define malware/retention controls before wider use.
- Do not expose admin shared secrets or use source fallback credentials. Review administrator identity, access logging and credential rotation.
- Measure polling load: three-second location/tracking requests and separate notification polling scale with active sessions. Add monitoring, rate controls and capacity tests before growth.
- Google API billing, route quotas, key restrictions, SMTP delivery, worker supervision, backup restores and payment-provider processing require environment-specific verification.
- Installed versions, migration practices, development web serving and file-session storage need a documented production deployment plan.
- Interactive public API documentation is disabled in the current API configuration. Use the route inventory and source definitions for integration; do not assume an old README /docs link is active.

17 Data relationships and model catalog



The following catalog covers every SQLAlchemy persistence model found in backend/models.py. Oracle column type labels are taken from the exported live DDL where available. PK and FK indicators come from the model definitions; the SQL export remains the authority for constraint names, checks, unique keys and identity clauses.

Orders retain item and total snapshots. Driver and restaurant progress is tracked through delivery status history. Latest GPS is separate from the order's item data. System configuration holds pricing and policy/refund records; those JSON/value payloads must not be treated as free-form unaudited financial storage.

The inspected model catalog and live DDL contain 25 tables and 160 columns. No customer rows, passwords, tokens or live coordinates are included in this document.

EVENT_OUTBOX

Transactional events awaiting external Kafka or SMTP delivery.

Column	Oracle type	Rule	Foreign-key target
event_id	VARCHAR2(36 CHAR)	Required	-
event_type	VARCHAR2(60 CHAR)	Required	-
payload	CLOB	Required	-
published	NUMBER(*,0)	Required	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

PROMOTIONS

Promotion configuration and eligibility attributes.

Column	Oracle type	Rule	Foreign-key target
title	VARCHAR2(120 CHAR)	Required	-
description	CLOB	Required	-
code	VARCHAR2(40 CHAR)	Required	-
percent	NUMBER(*,0)	Required	-
minimum	NUMBER(12,2)	Required	-
cap	NUMBER(12,2)	Required	-
first_order_only	NUMBER(*,0)	Required	-
enabled	NUMBER(*,0)	Required	-
starts_at	DATE	NULL	-
ends_at	DATE	NULL	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

SYSTEM_CONFIG

Key/value operational settings, pricing, cancellation snapshots and refund request payloads.

Column	Oracle type	Rule	Foreign-key target
key	VARCHAR2(100 CHAR)	PK	-
value	VARCHAR2(2000 CHAR)	Required	-

USERS

Authentication identity, role and account-level preferences.

Column	Oracle type	Rule	Foreign-key target
email	VARCHAR2(254 CHAR)	Required	-
password	VARCHAR2(255 CHAR)	Required	-
role	VARCHAR2(20 CHAR)	Required	-
name	VARCHAR2(120 CHAR)	Required	-
phone	VARCHAR2(30 CHAR)	NULL	-
token_version	NUMBER(*,0)	Required	-
order_sound_enabled	NUMBER(1,0)	Required	-
id	NUMBER(*,0)	PK	-

AUDIT_EVENTS

Actor, action, target and operational audit details.

Column	Oracle type	Rule	Foreign-key target
actor_id	NUMBER(*,0)	NULL	users.id
action	VARCHAR2(80 CHAR)	Required	-
target	VARCHAR2(160 CHAR)	Required	-
details	CLOB	Required	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

CUSTOMERS

Customer-specific profile linked to a user.

Column	Oracle type	Rule	Foreign-key target
id	NUMBER(*,0)	PK	users.id
order_mode	VARCHAR2(20 CHAR)	Required	-

DRIVERS

Delivery partner availability and vehicle profile.

Column	Oracle type	Rule	Foreign-key target
id	NUMBER(*,0)	PK	users.id
is_online	NUMBER(*,0)	Required	-
vehicle_type	VARCHAR2(80 CHAR)	NULL	-
vehicle_number	VARCHAR2(80 CHAR)	NULL	-

FILES

Uploaded file ownership and stored path metadata.

Column	Oracle type	Rule	Foreign-key target
user_id	NUMBER(*,0)	Required	users.id
purpose	VARCHAR2(30 CHAR)	Required	-
entity_id	NUMBER(*,0)	NULL	-
path	VARCHAR2(1000 CHAR)	Required	-

Column	Oracle type	Rule	Foreign-key target
mime	VARCHAR2(80 CHAR)	Required	-
id	NUMBER(*,0)	PK	-

PASSWORD_RESETS

Expiring password-reset token records.

Column	Oracle type	Rule	Foreign-key target
user_id	NUMBER(*,0)	Required	users.id
digest	VARCHAR2(64 CHAR)	Required	-
expires_at	DATE	Required	-
used	NUMBER(*,0)	Required	-
id	NUMBER(*,0)	PK	-

RESTAURANTS

Business identity, address, availability and preparation information.

Column	Oracle type	Rule	Foreign-key target
id	NUMBER(*,0)	PK	users.id
name	VARCHAR2(120 CHAR)	Required	-
description	VARCHAR2(1000 CHAR)	NULL	-
cuisine	VARCHAR2(80 CHAR)	Required	-
kind	VARCHAR2(20 CHAR)	Required	-
address	VARCHAR2(500 CHAR)	NULL	-
is_open	NUMBER(*,0)	Required	-
opening	VARCHAR2(5 CHAR)	Required	-
closing	VARCHAR2(5 CHAR)	Required	-
delivery_minutes	NUMBER(*,0)	Required	-

ADDRESSES

Saved customer addresses and default selection.

Column	Oracle type	Rule	Foreign-key target
customer_id	NUMBER(*,0)	Required	customers.id

Column	Oracle type	Rule	Foreign-key target
label	VARCHAR2(80 CHAR)	Required	-
details	VARCHAR2(500 CHAR)	Required	-
is_default	NUMBER(1,0)	Required	-
place_id	VARCHAR2(255 CHAR)	NULL	-
id	NUMBER(*,0)	PK	-

CUSTOMER_LOCATIONS

Latest customer location information.

Column	Oracle type	Rule	Foreign-key target
customer_id	NUMBER(*,0)	PK	customers.id
latitude	FLOAT(126)	Required	-
longitude	FLOAT(126)	Required	-
address	VARCHAR2(500 CHAR)	NULL	-
updated_at	DATE	Required	-

DRIVER_LOCATIONS

Latest driver GPS fix, heading and update timestamp.

Column	Oracle type	Rule	Foreign-key target
driver_id	NUMBER(*,0)	PK	drivers.id
latitude	FLOAT(126)	Required	-
longitude	FLOAT(126)	Required	-
heading	FLOAT(126)	NULL	-
updated_at	DATE	Required	-

MENU_ITEMS

Restaurant menu catalog, prices and active availability.

Column	Oracle type	Rule	Foreign-key target
restaurant_id	NUMBER(*,0)	Required	restaurants.id
name	VARCHAR2(120 CHAR)	Required	-
description	VARCHAR2(1000 CHAR)	NULL	-

Column	Oracle type	Rule	Foreign-key target
category	VARCHAR2(80 CHAR)	NULL	-
price	NUMBER(12,2)	Required	-
veg	NUMBER(*,0)	Required	-
available	NUMBER(*,0)	Required	-
id	NUMBER(*,0)	PK	-

ORDERS

Order owner, restaurant, assignment, mode, totals and lifecycle snapshot.

Column	Oracle type	Rule	Foreign-key target
customer_id	NUMBER(*,0)	Required	customers.id
restaurant_id	NUMBER(*,0)	Required	restaurants.id
driver_id	NUMBER(*,0)	NULL	drivers.id
request_key	VARCHAR2(64 CHAR)	Required	-
status	VARCHAR2(40 CHAR)	Required	-
mode	VARCHAR(20)	Required	-
address	VARCHAR2(500 CHAR)	Required	-
total	NUMBER(12,2)	Required	-
tip	NUMBER(12,2)	Required	-
tax	NUMBER(12,2)	Required	-
service_fee	NUMBER(12,2)	Required	-
delivery_fee	NUMBER(12,2)	Required	-
discount	NUMBER(12,2)	Required	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

PLATFORM_CONTENT

Managed platform content records.

Column	Oracle type	Rule	Foreign-key target
key	VARCHAR2(80 CHAR)	PK	-
title	VARCHAR2(120 CHAR)	Required	-

Column	Oracle type	Rule	Foreign-key target
description	CLOB	Required	-
enabled	NUMBER(*,0)	Required	-
media_file_id	NUMBER(*,0)	NULL	files.id
updated_at	DATE	Required	-

RESTAURANT_PRESENTATIONS

Restaurant display and presentation configuration.

Column	Oracle type	Rule	Foreign-key target
restaurant_id	NUMBER(*,0)	PK	restaurants.id
cover_file_id	NUMBER(*,0)	NULL	files.id
logo_file_id	NUMBER(*,0)	NULL	files.id
gallery_file_ids	CLOB	Required	-
busy_mode	VARCHAR2(20 CHAR)	Required	-
busy_until	DATE	NULL	-
prep_extra_minutes	NUMBER(*,0)	Required	-
capacity	NUMBER(*,0)	Required	-
updated_at	DATE	Required	-

CART_ITEMS

Customer cart quantities and selected menu references.

Column	Oracle type	Rule	Foreign-key target
customer_id	NUMBER(*,0)	Required	customers.id
menu_item_id	NUMBER(*,0)	Required	menu_items.id
special_instructions	VARCHAR2(1000 CHAR)	NULL	-
quantity	NUMBER(*,0)	Required	-
id	NUMBER(*,0)	PK	-

DELIVERY_STATUS

Append-only status timeline used for preparation and delivery tracking.

Column	Oracle type	Rule	Foreign-key target
order_id	NUMBER(*,0)	Required	orders.id
status	VARCHAR2(40 CHAR)	Required	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

NOTIFICATIONS

Recipient-specific order and operational notification records.

Column	Oracle type	Rule	Foreign-key target
user_id	NUMBER(*,0)	Required	users.id
order_id	NUMBER(*,0)	NULL	orders.id
kind	VARCHAR2(40 CHAR)	Required	-
body	VARCHAR2(500 CHAR)	Required	-
read_at	DATE	NULL	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

ORDER_ITEMS

Purchased item names, quantities and price snapshots.

Column	Oracle type	Rule	Foreign-key target
order_id	NUMBER(*,0)	Required	orders.id
menu_item_id	NUMBER(*,0)	Required	menu_items.id
name	VARCHAR2(120 CHAR)	Required	-
special_instructions	VARCHAR2(1000 CHAR)	NULL	-
quantity	NUMBER(*,0)	Required	-
price	NUMBER(12,2)	Required	-
id	NUMBER(*,0)	PK	-

ORDER_MESSAGES

Order-related communication records.

Column	Oracle type	Rule	Foreign-key target
order_id	NUMBER(*,0)	Required	orders.id
user_id	NUMBER(*,0)	Required	users.id
body	CLOB	Required	-
created_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

RATINGS

Numeric feedback associated with completed orders.

Column	Oracle type	Rule	Foreign-key target
order_id	NUMBER(*,0)	Required	orders.id
restaurant	NUMBER(*,0)	Required	-
driver	NUMBER(*,0)	NULL	-
id	NUMBER(*,0)	PK	-

REVIEWS

Text feedback associated with completed orders.

Column	Oracle type	Rule	Foreign-key target
order_id	NUMBER(*,0)	Required	orders.id
customer_id	NUMBER(*,0)	Required	customers.id
text	VARCHAR2(2000 CHAR)	Required	-
id	NUMBER(*,0)	PK	-

SUPPORT_TICKETS

Customer/partner support cases and responses.

Column	Oracle type	Rule	Foreign-key target
user_id	NUMBER(*,0)	Required	users.id
order_id	NUMBER(*,0)	NULL	orders.id
subject	VARCHAR2(120 CHAR)	Required	-
description	CLOB	Required	-
status	VARCHAR2(20 CHAR)	Required	-

Column	Oracle type	Rule	Foreign-key target
resolution	CLOB	Required	-
created_at	DATE	Required	-
updated_at	DATE	Required	-
id	NUMBER(*,0)	PK	-

18 API inventory and source references

The inventory below is generated from the current FastAPI application's registered routes. Business endpoints use the /api prefix; the service health route is /health. Authentication and role requirements must be checked in the route dependencies and service ownership checks before integrating a client.

Customer and driver browser code normally calls same-origin Django routes, which proxy to this internal API. Do not put API bearer tokens or admin secrets into public JavaScript.

Source references: backend/main.py; backend/models.py; backend/schemas.py; backend/routers; backend/services; backend/worker.py; frontend/config/settings.py; frontend/common_app; frontend/static; script/dashvanti-services.sh; database/oracle_schema.sql; requirements.txt.

The root README includes older scaffold assumptions (including container setup and feature limitations). When it conflicts with the inspected source/runtime, use the current code and this dated runbook, then update the documentation as part of the next release.

Methods	Registered path
GET	/health